Automatic Configuration of Simulation Environments for Human-Robot Interaction in Highly-Customizable Healthcare Scenarios

Alberto Boffi
Politecnico di Milano, Italy
alberto2.boffi@mail.polimi.it

Abstract

The evolution of robot operating environments towards personal assistance scenarios leads to new development challenges. The interaction with human entities translates into intrinsically stochastic models, requiring professional figures to refine the customizable portion of the scenario until the resulting probability of success is acceptable. This paper describes the design and implementation of a support system aimed at the automatic configuration of the virtual simulation environment, starting from the user's specifications. Results are then validated on significant use cases. The last portion of the paper opens the way to possible future developments, in the direction of an automatic setup of the robot controller through reinforcement learning techniques.

1 Introduction

The operating environment of robots is more than ever in constant evolution. Nowadays, robotics applications are expanding in the area of personal assistance and human support [5, 4, 9]. Against classical industrial settings, human interaction requires a more critical development [14, 7], since the environment is characterized by a strong stochastic component capturing the variability of the psychological factor [6].
Previous works already faced this challenge by designing a model-driven approach, based on Hybrid Automata [2], for the creation of these scenarios, along with a formal verification method performed through Statistical Model Checking [11, 12, 1].
The focus is mainly referred to the healthcare domain, the most common and useful for such kind of study [10], although results may be easily generalized to several human-robot interaction contexts.

The target configuration is made up of human entities and battery-powered mobile robots, interacting in a well defined environment, like a hospital floor. The mission of each robot is to successfully serve all the assigned humans in the given scenario, characterized by the specific human needs. The same scenario can be customized by the final user, who is generally a professional figure without any technical knowledge related to formal models, and can therefore set and modify different key parameters through a high-level representation.
When playing in a non-deterministic environment, the outcome of the verification process is an estimation of the probability of successfully completing the mission. Based on these results, the user can refine the model until the outcoming probability is satisfactory.
The design-time phase is then supported by a deployment stage consisting in a model-to-code mapping that allows simulation in a realistic virtual environment, ensuring that the deployment system actually matches the formal property of the designed model [13].
In this context, simulation is of considerable relevance in order to validate the outcome of the

verification process, allowing the final user to face the result of the set configuration. For this reason, there's the need for the user to immediately visualize the effects of his settings, so as to eventually refine them in an iterative calibration process that is efficient and productive.

The main contribution of this work is to produce a valid way to automatically map the user-customizable parameters into the final simulation, to support any professional figure in the tuning process of the scenario. The research aims at flexible approaches that can be adapted to any simulation environment, always placing the efficiency of the operators in foreground.

2 Background

The system as-is can be seen as the result of a simulation run through the CoppeliaSim platform, supported by an agnostic controller written in Python. They both lie on a ROS-based middleware layer, used by the controller to receive data and communicate with the simulation through Remote API [13].

The simulation is identified by a scene, a packet collecting all the objects interacting in the environment [17]. Every object is associated with:

- Static properties: observable when the object is not performing any action. They include attributes as type, size, position, name or color;
- Simulation script: Lua embedded script defining the run-time behavior of the object, observable once the simulation is launched [17, 3]. Lua is a dynamic and lightweight scripting language, executed through an integrated interpreter. It supports both functional and object-oriented programming [8].

The user can customize any kind of data about the scene, including all the existing objects, their static properties and attributes regarding the run-time performances. This possibility is essential to allow realistic virtual reconstructions of real environments where robots and humans must interact.

It's anyway pivotal that the development process described in this paper is designed to adapt to any scene and whatever the choice of the portion to be customized.

2.1 Requirements Analysis

The goal of the system to-be is to map the user-customizable parameters into the final simulation, by building the entire simulation environment and defining the behavior of its entities, within a completely automatic process. Once the scenario is refined from a high level interface, the system must be able to reflect the preferences in the resulting simulation, without any further interaction from the user side.

Some domain assumptions:

- Parameter values are already stored into a suitable data representation format: we are always referring to the user settings through a given data file. There are several possible upstream data entry processes leading to the file generation, allowing to deal with correct and consistent data, which are not the subject of this work. We especially assume to work with JSON extension, mainly due to the wide expressive power guaranteed despite the simplicity of the language [15]. This assumption will be reconsidered only at the end of the paper, discussing possible future developments;

- The whole simulation environment lies on a single machine: both the controller and the simulation scene are placed in the same machine, where they can access the data file. Any kind of network management in the context of a multi-machine scenario deviates from the purpose of this paper.

To ensure the achievement of the goal, the design is based upon some main requirements:

- The system must access the data file and understand information contained;
- The system must add objects within the scene;
- The system must access objects of the scene by setting their properties;
- The system must access objects in the scene by setting their run-time parameters;
- The system must allow the controller to communicate with the simulation;
- The system must start running automatically.

3 Architecture

The controller side portion of the system can be easily implemented by parsing the data file and allowing the controller to store values into a suitable dictionary data structure, so as to use the corresponding information to dynamically interact with the simulation [20]. The challenge of the design and implementation phase is to entirely build a scene that faithfully reflects the set configuration, by dealing with the dependencies of the CoppeliaSim platform in use.

The ambitious freedom of customization prevents us from acting within the simulation script, where the behavior described comes into play only once the environment is fully defined [3].

The workflow can be thus represented by a three-tier architecture, which sees the data file as the first layer, and the simulation scene as the last. What falls between the two is a compound middle-layer with the main role of reading data, manipulating them, managing objects of the scene and creating an interface between the data and the simulation. The main design

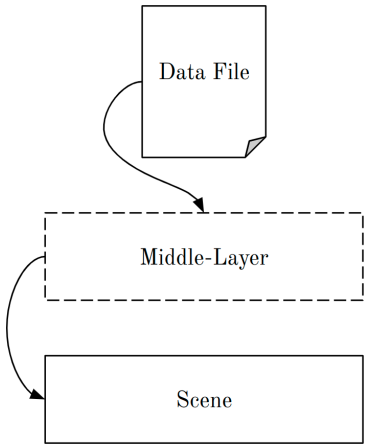


Figure 1: Three-tier architecture.

consideration consists in identifying the nature of the middle layer. CoppeliaSim offers several coding approaches to develop around the simulator [17, 3]:

3.1 Network-based methods

Network-based methods includes Remote API, a cross-platform strategy supporting bidirectional data-streaming, currently used by the controller; ROS node, which provides several interfaces connecting the simulator to another machine in the same ROS environment; TCP/IP (ZeroMQ...) node, similar to the previous method, where the interaction is managed by different communication protocols. Such techniques have been discarded, as the management of a network framework does not match the system requirements.

3.2 Plugins

Writing a plugin for CoppeliaSim means writing a shared library in C or C++ with the aim of extending CoppeliaSim functionality, or possibly exploiting the benefits of a compiled language. None of the goals is shared by the system to-be.

3.3 Add-ons

Unlike plugins, the purpose of an add-on is to support and interact with the scene, without extending the functionality of the simulator. Add-ons are Lua scripts that start running in the background when the simulator is launched, persistent along all open scenes and not meant to be restricted to the specific simulation.

A sandbox script is a similar alternative, mainly used to exploit the input-code insertion feature given by the simulator interface, not satisfying the rigid constraint relative to the independence of the mapping process to any human interaction.

Another analogous alternative is represented by the Customization Script. A Customization Script is however, as the Simulation Script, an embedded script editable via the platform interface, aimed at simple purposes and not addressed to the implementation of complex systems.

The use of a pure add-on, on the other hand, perfectly serves the goal of the system to-be. Add-ons provides all the skills needed to build an agnostic mechanism that meets the system requirements [17, 3, 8]:

- A Lua script can easily access the file system, allowing the add-on to acquire and parse the data file;
- CoppeliaSim provides several API functions to access any aspect of a scene, from associated or unassociated code, offered through different syntax for specific programming languages, Lua included;
- CoppeliaSim provides a large number of communication and exploitable interfacing methods to exchange data between the simulation and external code. They range from signal and data blocks to Remote API, WebSocket or serial ports. The most clear and plain methods target the add-ons, as this is the main purpose for which they're conceived;
- An add-on can be automatically executed at the program startup, without needing to be manually triggered. It supports command-line arguments.

3.4 Middle-layer

The architecture of the middle-layer is based upon several units operating at two different execution phases.

3.4.1 Static phase

The static phase is directed by the entire add-on code, where each component plays an active role. The first stage is performed by the DataManager component. DataManager provides the operations needed to retrieve the user configuration of the scenario, by allowing to read the data file, parse it and encode it in a suitable Lua table used by the next components to access needed parameters, as well as during the run-time phase to exchange data.

The second stage is carried out by the manager of each class of objects in the scenario, regardless of its cardinality and nature. Handled functions are:

- Create the object;
- Set the static properties of the object;
- Associate a simulation script to the object, providing the interface used by the scene to retrieve run-time parameters during the execution phase.

Each task is accomplished by interfacing with the DataManager, as shown in Fig. 3.2. The first two tasks can be performed by means of some of the many offered local API functions [3], while the last one deserves a short crucial analysis: for each object of the class managed by the component under discussion, the simulation script is the same. Values of each customizable variable are indeed not directly assigned, as they reference the specified interface between the scene and the add-on.

The reason for the dynamic assignment of the script lies in the configuration of the first and only line of the code, where the object is referenced to the correct interface, i.e. to the corresponding data among all those of every element of the class.

Operating in this context, we are able to provide the interfacing mechanism using functions offered by CoppeliaSim to pack, write, read and unpack data interpreted as custom data blocks [3].

Listing 1: Code 3.1: Robot rob1 interfaces with the add-on.

```
robot = sim.unpackTable(sim.readCustomDataBlock(
    sim.handle_scene, "scenario"
)).robots.rob1
```

Listing 2: Code 3.2: Example of dynamic value assignment.

```
v_max = robot["v_max"]
```

The static phase is executed the first time when the simulator is opened, getting the name of the data file as a parameter. The following times the add-on can be easily run again by resetting it through the user interface: at each reset, the DataManager component repeats its tasks fetching any changes in the user data, and basic static properties (such as the position) are adjusted accordingly as well. The simulation script remains unchanged, as independent from attribute values.

3.4.2 Run-time phase

Results statically produced are then used during the run-time phase, starting at the simulation launch and completing the whole process performed by the system. The data-encoding implemented by the DataManager and the interface provided through the assignment of the simulation script are exploited by each scene object, retrieving the corresponding dynamic values by interfacing with AddOnManager, the main component of the add-on.

The run-time phase represents an important choice of the design process. Its presence is not necessarily mandatory to accomplish the goal, as run-time data can be directly encoded in the simulation script of every interested component, added during the static phase. However, the decision related to the insertion of a supplementary execution stage allows us to keep fixed simulation scripts, implying two main advantages:

- The first intuitive advantage derives from the negative effects of a partially defined code dynamically completed by another script, as less readability or understandability, and especially a more difficult maintenance process;
- The second one is purely practical, related to the modification of the data of the scenario from the user side: as discussed above, when resetting the add-on, the simulation script doesn't need any change or adjustment in order to fetch the newly inserted data.

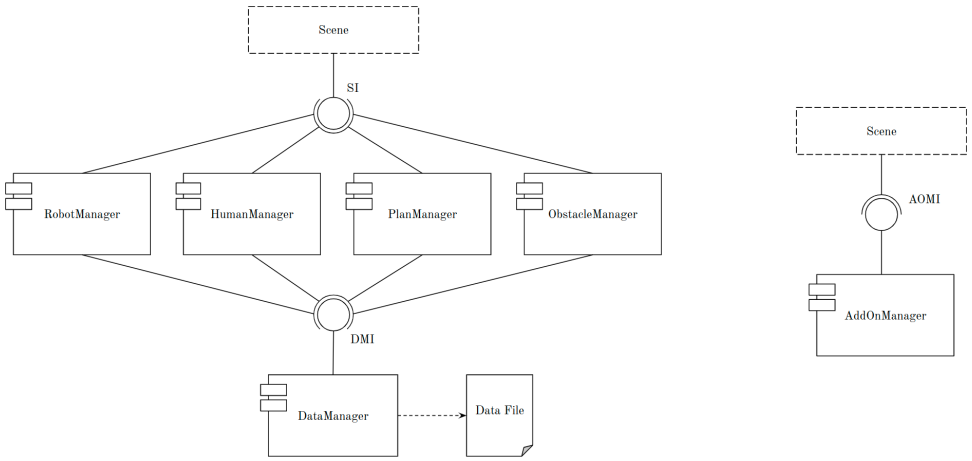


Figure 2: Component diagrams of the static (left) and run-time phases (right).

3.5 Data Representation Rules

Scenario parameters are currently encoded in a JSON data file. Rules and conventions related to the data representation are, however, widely untied to the adopted format as well as to any choice of the actual variables.

The design related to this phase aims at an intuitive and user-friendly portrayal of the full environment. Modeled entities are: robots, humans, the charging station shared by every robot, the floor plan and other obstacles, including cupboards and sofas. Some of the attributes characterizing them are shown in [11], pp. 308, Fig. 3. Non-unique elements without a run-time behavior are collected in simple lists, while "active" entities such as robots and humans are stored as objects identified by a specific name, defined by the user and used in the simulation script to address the entity itself, as in the assignment of the corresponding interface. This approach truly simplifies the control of the data as the number of elements increases.

The environment is configured in a two dimensional x-y plane where the height information is ignored, playing an useless role for our purposes. The set coordinates of an element are interpreted as the position of its center of gravity with respect to the center of the environment. They are expressed in decameters by constraint of the platform [3]. The floor plan is made up by a set of areas, or rooms, each one denoted by an ordered list of at least two points: each pair of consecutive points describes a wall delimiting a side of the corresponding area.

Some assumptions:

- A wall is always oriented parallel to either the x or the y axis;
- If the pair of points $(p[i], p[i+1])$ denotes an horizontal wall, the possible following pair $(p[i+1], p[i+2])$ must identify a vertical wall, or vice versa. Two consecutive walls A and B with the same orientation can always be replaced by a single block C with length $l[C] = l[A] + l[B]$;
- The walls of every area have a fixed (but editable) thickness of 1.5 meters, i.e. the size relative to the opposite axis with respect to its orientation. The thickness must be kept in consideration when encoding the planimetry, as it plays a critical role in fitting walls orthogonal to each other: to avoid overlaps, each wall covers its starting point, while the landing point is covered by the following wall. In a block parallel to the abscissa (ordinate) axis, the $y(x)$ coordinate of the point is referred to the middle of the wall, at 0.75 meters from its border.

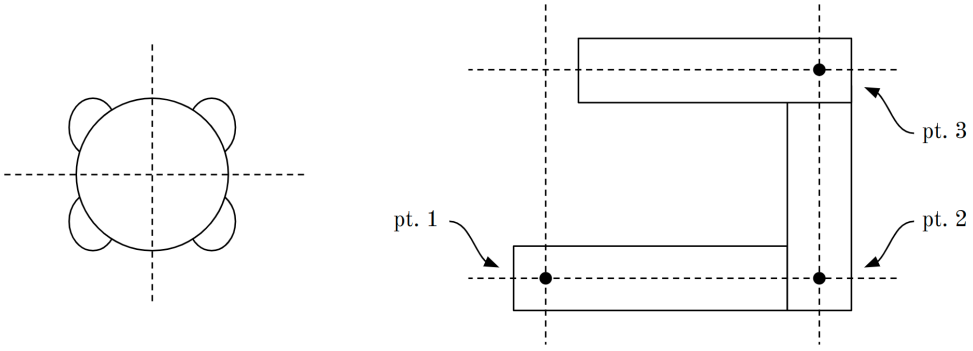


Figure 3: Minimal technical draftings of a robot and a three-wall-delimited area.

Listing 3: Code 3.3: JSON structure of the main entities.

```
"robot": {
    "v_max": n,
    "a_max": n,
    "served_humans": ["hum1", "hum2"],
    "start": {
        "x": n,
        "y": n
    },
    "battery": {
        "c_start": n,
        "t_chg": n,
        "t_dchg": n
    }
}

"human": {
    "v": n,
    "p_f": "YoungHealthy",
    "p": "HumanLeader",
    "start": {
        "x": n,
        "y": n
    },
    "dest": {
        "x": n,
        "y": n
    }
}

"charging_station": {
    "x": n,
    "y": n
}

"walls": [
    [[x,y], [x,y], [x,y], [x,y], [x,y]],
    [[x,y], [x,y], [x,y], [x,y], [x,y]]
]

"generic_obstacles": [[x,y], [x,y], [x,y], [x,y], [x,y]]
```

4 Implementation

The parsing of the data file is performed by an external library [16], which is the only third-party component of the software. The management of each entity is distributed into dedicated modules, eventually collected and used by the main add-on file being launched. The main file stores the full implementation of the Data Manager, while components handling every class of items are split between the corresponding module, where the item is defined, and the main file, where needed methods are called.

This project structure is specially meant to be easily scalable and robust, allowing low maintenance times. The scalability of the system pushed in the direction of a top-down implementation, from the creation of the main source to its enrichment by complementary modules, whenever needed. This approach methodology was also reflected in the small portion of each created

component.

5 Experimental Results

The work has been tested on significant use cases. Operating mainly in healthcare scenarios, the system is validated through environments reflecting the most typical situations observable inside a hospital building. The static results of the analysis, along with their related circumstances, are shown below. The objective is to verify the correctness of the building process performed by the static phase of the system, in response to the current configuration.

- **Regular hospital plan**: The first use case is addressed to a typical structure of a hospital plan. The environment includes four rooms of the same size near a large open space. Each room contains one or two patients with their relative robots, while the robots density in the open space is much lower as humans can be generally considered both patients and medical staff. The furniture is arranged as expected in a classical configuration, and the charging station is easily accessible by any robot.

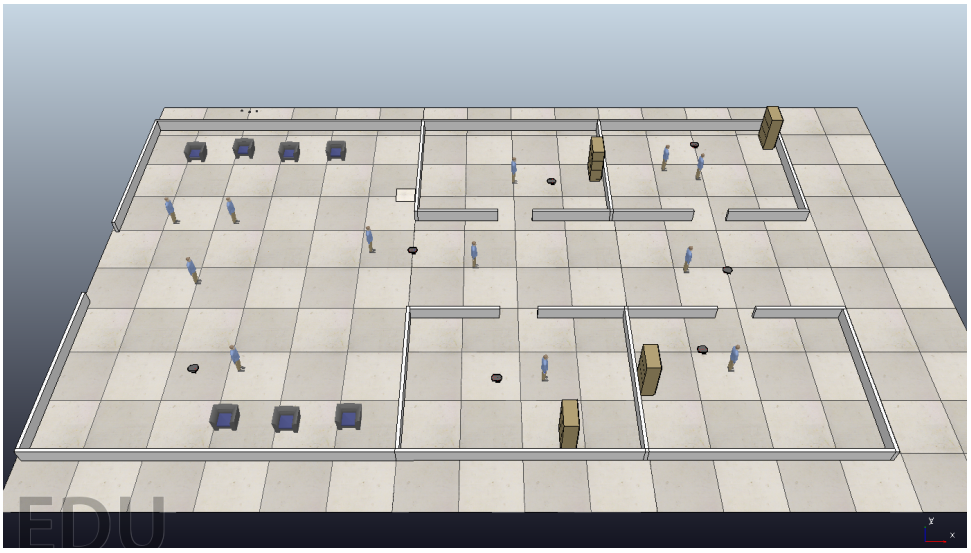


Figure 4: Regular hospital scenario.

- **Overcrowded hospital plan**: In case of war, natural disasters, or at the outbreak of epidemic as COVID-19, public hospitals get easily overcrowded [18]. In this scenario, it is common to have patients occupying areas actually designed for circulation reasons (as hallways), or for several other purposes. This also means that even doctors and medical equipment needs to be moved, representing obstacles the robots must deal with while accomplishing their tasks. The following configuration tries to illustrate this situation: the environment contains one main hallway developing on multiple sides and housing the charging station, along with three separate rooms. Each room is fulfilled in respect to its capacity, and the hallway is occupied as well (sofas can also be generalized to any kind of medical equipment). Robots are insufficient compared to the number of humans they have to serve, as expected in emergency situations.

- **Emergency room**: The structure of an emergency department differs from the previous ones due to the presence of a unique big area where patients are placed, and separated by suitable dividers. Also in this case, especially in critical situations, the use of personal assistant robots can be really helpful for both patients and medical staff. Dividers can be

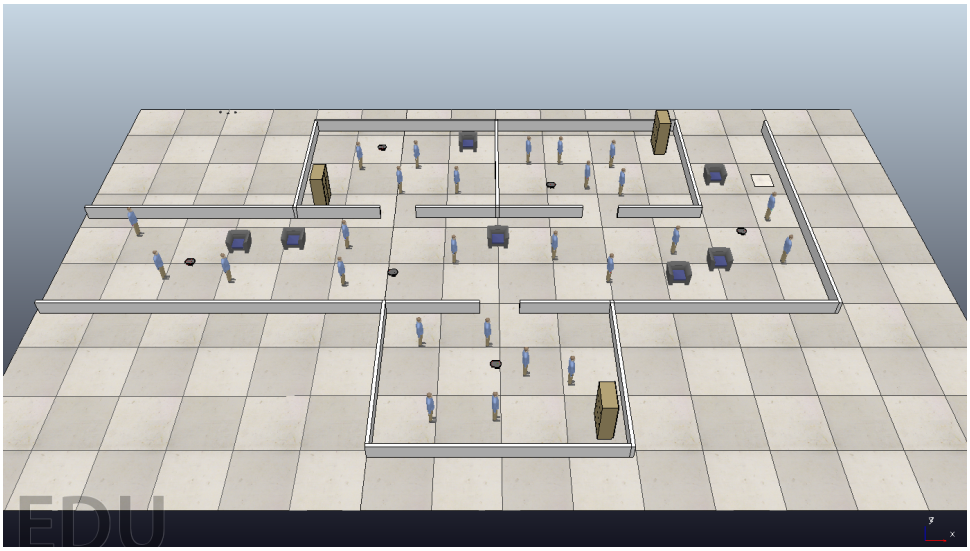


Figure 5: Overcrowded hospital scenario.

modeled by walls, and all the entities of the environments are placed according to regular conditions.

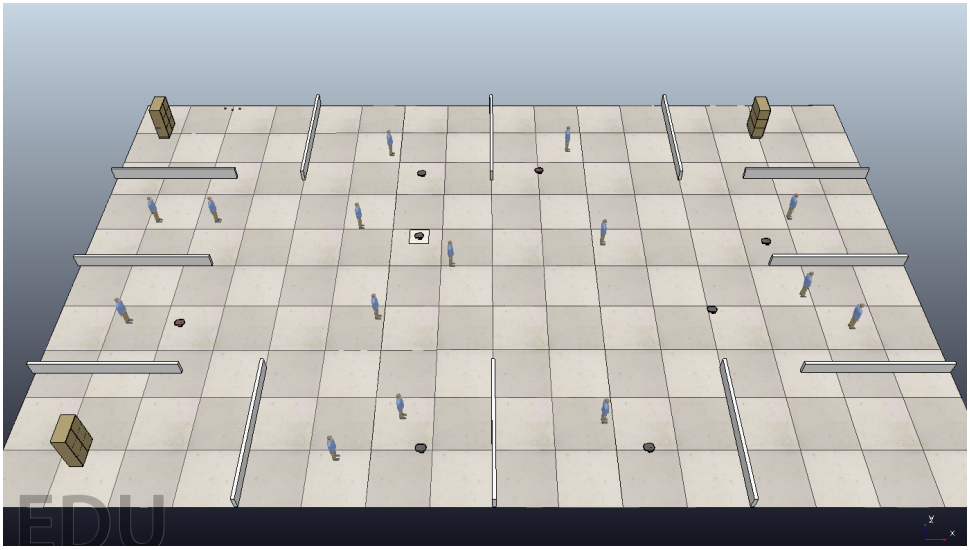


Figure 6: Emergency room scenario.

6 Control Robots Configuration through Reinforcement Learning

Relaxing foreground domain assumptions can lead to several future developments. The supplementary step presented at this point of the research proposes a possible direction for the process of the creation of the data file. The aim is to present a very simple general approach for configuring robots through a reinforcement learning technique, further developable in separate study. Underlying motivations can be found in the research of completely autonomous processes needed to guarantee the most satisfying results in serving patients within the configured environment.

As widely discussed in [11] and [12], the workflow is based on three possible human-robot in-

teraction patterns. For the sake of simplicity, we just mention Human Follower and Human Leader frameworks. In the first one the robot moves towards a fixed destination followed by the human, while in the second case the robots must blindly follow the human. The algorithm shall therefore interact with the Hybrid Automata model that describes the behavior of the chosen pattern, while also facing the uncertainty introduced by it.

Before diving into the model, some observations:

- The most significant parameters affecting the robots performances are the maximum velocity and acceleration [11, 12]. For the clarity of modeling, we'll just fetch the velocity factor. Acceleration, as well as possible additional settings, can be anyway added preserving all the underlying principles.
- The floor can be ideally divided into a finite number of areas of the same size where the robot can be found at any instant. We can call these spaces "points", or "cells".
- The environment can be reduced, without loss of generality, by only considering the robot and the human to serve at that specific moment. All the other entities are evaluated as "obstacles".

The framework can be represented as a Markov Decision Process (S, A, P, R) [19]. The state space S contains all the ammissibile values of the tuple (p_r, p_h, v_r, v_h): $p_r(p_h)$ is an obstacle-free cell where the robot (human) is located at time instant t, and $v_r(v_h)$ is the velocity of the robot (human) calculated between instants $t-1$ and t. Note that, being constant, v_h can be omitted. However, it is useful in describing the general concept that applies even in the more realistic case, where the velocity can change over time. The pairs (p_r, v_r) and (p_h, v_h) are estimators of the possible new position of the relative entity in the immediate future, so that the whole state can estimate the distance between the two.

Let now $(p_r(t), v_r(t), p_h(t), v_h(t)) \rightarrow (p_r(t'), v_r(t'), p_h(t'), v_h(t'))$ with $t' = t + 1$, be a generic transition of the action space A. The values of $p_r(t')$ given $p_r(t)$ are driven by the Hybrid Automata. The parameter $v_r(t')$ is what we are interested in adjusting, and can be calculated as $v_r(t') = v_r(t) \pm \Delta$ for an appropriate choice of the regulation factor Δ. The stochastic transition model $P : S \times A \times S \rightarrow [0, 1]$ is determined by the automata.

In the Human Leader pattern, the reward function R is the mere result of studying the mutual position of the human and the robot. It can be designed and adjusted in several ways, according to the resulting performances. In the Human Follower settings, R must also take into consideration the destination. In general, it can be expressed as the weighted sum of different reward evaluations: $w_1 R_1 + w_2 R_2$, $w_1 + w_2 = 1$. The discount factor and the initial state distribution are outside the scope of this analysis.

Function R_1, as for the Human Leader case, takes into consideration the distance from the human, while R_2 judges the achievement of the destination, for instance by either a goal-reward or action-penalty schema. The problem can be then approached by several techniques able to derive stochastic policies [19], which are the most suitable for the operating conditions discussed in this paper.

7 Conclusion

The realized system faced the challenge of providing an automatic method to fully configure a human-robot scenario aimed at simulation in a virtual environment, exploitable by professional figures typically operating in healthcare scenarios, such as experts in medical logistics and

workflow management. The design is based upon several strict requisites as perfect automation, flexibility, efficiency and full customization freedom, regarding features and behavior of any entity of the scenario. Versatility to any change and future evolution was particularly taken into consideration during the whole development activity. The system has been structured on a three-tier architecture, through a multi-phase add-on performing a bridge role between user parameters and the CoppeliaSim scene.
Results have been then tested among remarkable use cases, mainly referred to the healthcare domain. In the last step, we proposed a machine learning approach devoted to robot configuration, paving the way to possible future directions of the research.

References

[1] G. Agha and K. Palmskog. A survey of statistical model checking. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 28(1):1–39, 2018.

[2] R. Alur, C. Courcoubetis, N. Halbwachs, T. A. Henzinger, P. H. Ho, X. Nicollin, A. Olivero, J. Sifakis, and S. Yovine. The algorithmic analysis of hybrid systems. *Theoretical Computer Science*, 138(1):3–34, 1995.

[3] Coppelia Robotics. *CoppeliaSim User Manual*. Retrieved January 2022.

[4] M. M. de Graaf, S. Ben Allouch, and J. A. Van Dijk. Why would i use this in my home? a model of domestic social robot acceptance. *Human-Computer Interaction*, 34(2):115–173, 2019.

[5] S. García, D. Strüber, D. Brugali, T. Berger, and P. Pelliccione. Robotics software engineering: A perspective from the service robotics domain. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 593–604, November 2020.

[6] S. García, D. Strüber, D. Brugali, A. Di Fava, P. Schillinger, P. Pelliccione, and T. Berger. Variability modeling of service robots: Experiences and challenges. In *Proceedings of the 13th International Workshop on Variability Modeling of Software-Intensive Systems*, pages 1–6, February 2019.

[7] T. Gwon, H. Park, D. Seo, S. Lee, D. Kim, and S. Jeon. A study on safety evaluation criteria of the personal carrier robot based on ISO 13482. In *2019 Third IEEE International Conference on Robotic Computing (IRC)*, pages 477–482. IEEE, February 2019.

[8] R. Ierusalimschy, L. H. De Figueiredo, and W. C. Filho. Lua – an extensible extension language. *Software: Practice and Experience*, 26(6):635–652, 1996.

[9] T. Jacobs and G. S. Virk. ISO 13482 – the new safety standard for personal care robots. In *ISR/Robotik 2014; 41st International Symposium on Robotics*, pages 1–6. VDE, June 2014.

[10] M. Kyrarini, F. Lygerakis, A. Rajavenkatanarayanan, C. Sevastopoulos, H. R. Nambiappan, K. K. Chaitanya, A. R. Gani, J. Mathew, and F. Makedon. A survey of robots in healthcare. *Technologies*, 9(1):8, 2021.

[11] L. Lestingi, M. Askarpour, M. M. Bersani, and M. Rossi. Formal verification of human-robot interaction in healthcare scenarios. In *International Conference on Software Engineering and Formal Methods*, pages 303–324. Springer, Cham, September 2020.

[12] L. Lestingi, M. Askarpour, M. M. Bersani, and M. Rossi. A model-driven approach for the formal analysis of human-robot interaction scenarios. In *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 1907–1914. IEEE, October 2020.

[13] L. Lestingi, M. Askarpour, M. M. Bersani, and M. Rossi. A deployment framework for formally verified human-robot interactions. *IEEE Access*, 9:136616–136635, 2021.

[14] M. Luckcuck, M. Farrell, L. A. Dennis, C. Dixon, and M. Fisher. Formal specification and verification of autonomous robotic systems: A survey. *ACM Computing Surveys (CSUR)*, 52(5):1–41, 2019.

[15] T. Lv, P. Yan, and W. He. Survey on JSON data modelling. In *Journal of Physics: Conference Series*, volume 1069, page 012101. IOP Publishing, August 2018.

[16] T. Neylon. json.lua. GitHub Gist, October 2014. Retrieved from `https://gist.github.com/tylerneylon/59f4bcf316be525b30ab`.

[17] E. Rohmer, S. P. Singh, and M. Freese. CoppeliaSim (formerly V-REP): A versatile and scalable robot simulation framework. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1321–1326. IEEE, November 2013.

[18] J. Sammut. Why public hospitals are overcrowded: ten points for policymakers. 2009.

[19] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.

[20] G. Van Rossum and F. L. Drake Jr. *Python Tutorial*, volume 620. Centrum voor Wiskunde en Informatica, Amsterdam, The Netherlands, 1995.